PROGRAMACIÓN III · ANGULAR

Autorización por Roles

Angular + .NET · Apunte completo de estudio

Anatomía del JWT y claims · Rol como claim
roleGuard · *nglf por rol · Navbar dinámico

Instituto Técnico Superior Córdoba

Programación III · Prof. Mateo Alexis Vranich Prieto · Córdoba 2026

#	Sección
01	¿Por qué autorización por roles?
02	Anatomía del JWT: qué es y cómo se compone
03	Backend: agregar el rol al JWT
04	Backend: proteger endpoints por rol
05	Frontend: decodificar el JWT en Angular
06	Frontend: guard de roles
07	Frontend: mostrar y ocultar elementos según el rol

01 · ¿Por qué autorización por roles?

En el apunte anterior implementamos **autenticación**: el sistema verifica *quién sos* (login + JWT). Pero autenticación sola no alcanza. En cualquier sistema real hay usuarios con distintos niveles de acceso: un administrador puede crear y eliminar productos, mientras que un usuario común solo puede verlos.

La **autorización** responde a una pregunta distinta: ya sé quién sos, pero *¿qué podés hacer?* Esa decisión se basa en el **rol** del usuario.

> **Punto en común con el backend:** en .NET ya vimos `[Authorize]` para exigir autenticación y `[Authorize(Roles = "admin")]` para exigir un rol específico. Ahora vamos a implementar el mismo concepto del lado de Angular: el guard que ya tenemos va a aprender a distinguir roles.

Concepto	Pregunta que responde	Ejemplo
Autenticación	¿Quién sos?	Login con email y contraseña → JWT
Autorización	¿Qué podés hacer?	Solo admin puede eliminar productos

02 · Anatomía del JWT: qué es y cómo se compone

Antes de agregar el rol al token, necesitamos entender bien qué es un JWT y cómo está construido. Un **JWT** (JSON Web Token) es un string codificado que el servidor genera cuando el usuario hace login. Ese string viaja en cada request posterior como prueba de identidad.

Las tres partes del JWT

Un JWT es un string largo dividido en **tres partes** separadas por puntos. Cada parte está codificada en Base64, pero **no está encriptada** — cualquiera puede leer su contenido. Lo que lo hace seguro es la **firma**: si alguien modifica el contenido, la firma deja de coincidir y el servidor lo rechaza.

Estructura de un JWT

```
eyJhbGciOiJIUzI1NiJ9  .  eyJzdWIiOiIxIiwi...  .  xK9sD2mP
└──── HEADER ────┘        └──── PAYLOAD ────┘     └─ FIRMA ─┘
    (parte 1)                 (parte 2)           (parte 3)
```

Parte	Qué contiene	Ejemplo decodificado
1. Header	El algoritmo de firma usado y el tipo de token	{ "alg": "HS256", "typ": "JWT" }
2. Payload	Los claims: datos sobre el usuario (id, email, rol, expiración)	{ "sub": "1", "email": "ana@gmail.com", "role": "admin", "exp": 1724000000 }
3. Firma	El resultado de firmar Header + Payload con la clave secreta del servidor	HMACSHA256(header + payload, claveSecreta)

> **¿Por qué Base64 y no encriptado?** El JWT no necesita ser secreto — necesita ser **verificable**. Que el frontend pueda leer el payload (email, rol) es intencional: así puede mostrar el nombre del usuario o decidir qué botones mostrar. Lo que importa es que nadie pueda *modificar* el payload sin invalidar la firma. Solo el servidor tiene la clave secreta para firmar.

¿Qué son los claims?

Los claims son los pares clave-valor que van dentro del payload. Cada claim *afirma* algo sobre el usuario. Hay claims estándar (definidos por la especificación JWT) y claims personalizados que podés agregar según lo que necesite tu aplicación.

Claim	Clave	Qué representa
Subject	sub	Identificador del usuario (generalmente el Id)
Email	email	El email del usuario
Role	role	El rol del usuario (admin, user, etc.)
Expiration	exp	Fecha de expiración del token (Unix timestamp)
Issued At	iat	Fecha en que se generó el token

¿Cómo se genera el token en .NET?

Generación del JWT · 4 pasos

```
// Paso 1: Definir los claims (datos del usuario)
var claims = new List<Claim>
{
    new Claim(ClaimTypes.NameIdentifier, usuario.Id.ToString()),
    new Claim(ClaimTypes.Email, usuario.Email),
    new Claim(ClaimTypes.Role, usuario.Rol)
};

// Paso 2: Crear la clave de firma (desde appsettings.json)
var key = new SymmetricSecurityKey(
    Encoding.UTF8.GetBytes(config["Jwt:Key"]!)   // clave secreta
);
var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

// Paso 3: Armar el token con claims + expiración + firma
var token = new JwtSecurityToken(
    claims: claims,
    expires: DateTime.UtcNow.AddHours(8),
    signingCredentials: creds
);

// Paso 4: Serializar a string (el JWT final)
string jwt = new JwtSecurityTokenHandler().WriteToken(token);
// Resultado: "eyJhbGciOi...eyJzdWIiOi...xK9sD2mP"
```

¿Qué significa Bearer?

Cuando el cliente envía el JWT al servidor, lo incluye en el header `Authorization` con el prefijo `Bearer` (portador en inglés). Esto le dice al servidor: "el que porta este token tiene acceso". Es una convención del estándar HTTP.

Header de autorización

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWI...
               └ Prefijo ┘ └──── El JWT completo ────┘

// En Angular, el interceptor lo arma así:
const clonedReq = req.clone({
    setHeaders: {
        Authorization: `Bearer ${token}`   // "Bearer " + el JWT
    }
});

// En .NET, UseAuthentication() lee ese header automáticamente,
// valida la firma y, si es válido, extrae los claims.
```

> **Punto en común con el backend:** en .NET, `AddAuthentication(JwtBearerDefaults.AuthenticationScheme)` configura al servidor para que busque el token en el header `Authorization: Bearer`. En Angular, el interceptor se encarga de agregarlo a cada request. Los dos lados hablan el mismo "idioma" gracias al estándar Bearer.

El ciclo de vida completo del JWT

Paso	Quién	Qué hace con el JWT
1	**Usuario**	Envía email y contraseña (no hay JWT todavía)
2	**API .NET**	Valida credenciales, crea los claims, firma el JWT y lo retorna
3	**Angular**	Recibe el JWT y lo guarda en localStorage
4	**Angular (interceptor)**	En cada request, agrega Authorization: Bearer + JWT
5	**API .NET**	Lee el header, valida la firma, extrae los claims
6	**API .NET**	Usa los claims para decidir: ¿tiene permiso? (rol, expiración)

03 · Backend: agregar el rol al JWT

Ahora que entendemos la estructura del JWT, vamos a modificar el backend para que incluya el **rol** del usuario como un claim en el payload. El proceso tiene tres pasos: agregar el campo Rol a la entidad, generar la migración y modificar el método que genera el token.

Paso 1: agregar la propiedad Rol a la entidad Usuario

Models/Usuario.cs

```
public class Usuario
{
    public int Id { get; set; }

    [Required]
    [EmailAddress]
    public string Email { get; set; }

    [Required]
    public string PasswordHash { get; set; }

    [Required]
    public string Rol { get; set; } = "user";   // valor por defecto
}
```

El campo `Rol` es un string simple. Los valores más comunes son "user" y "admin". Al tener un valor por defecto, todo usuario nuevo se registra como "user" a menos que se indique lo contrario.

Paso 2: generar la migración

terminal

```
dotnet ef migrations add AgregarRolAUsuario
dotnet ef database update
```

Esto agrega la columna `Rol` a la tabla Usuarios en la base de datos. Los usuarios existentes van a tener el campo vacío — podés actualizar los que necesites directamente en la BD o crear un endpoint de administración.

Paso 3: incluir el rol como claim en el JWT

Cuando el usuario hace login, el servidor genera el JWT. Ahora, además del email y el id, incluimos el rol del usuario como un claim de tipo `ClaimTypes.Role`. Este es el claim estándar que `[Authorize(Roles)]` lee automáticamente.

Services/AuthService.cs — método que genera el JWT

```
private string GenerarToken(Usuario usuario)
{
    var claims = new List<Claim>
    {
        new Claim(ClaimTypes.NameIdentifier, usuario.Id.ToString()),
        new Claim(ClaimTypes.Email, usuario.Email),
        new Claim(ClaimTypes.Role, usuario.Rol)   // <-- EL ROL
    };

    var key = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(_config["Jwt:Key"]!)
    );
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
        claims: claims,
        expires: DateTime.UtcNow.AddHours(8),
        signingCredentials: creds
    );

    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

> **Importante:** el código es casi idéntico al de la sección 2, pero ahora incluye la línea `new Claim(ClaimTypes.Role, usuario.Rol)`. Esa sola línea es la diferencia entre un JWT que solo identifica al usuario y uno que además dice *qué puede hacer*. Recordá que si alguien modificara el rol en el token, la firma dejaría de ser válida.

04 · Backend: proteger endpoints por rol

Con el rol ya incluido en el JWT, proteger endpoints específicos es una sola línea. El atributo `[Authorize(Roles = "admin")]` hace que ASP.NET Core lea el claim `ClaimTypes.Role` del token y lo compare automáticamente.

Controllers/ProductosController.cs

```
[ApiController]
[Route("api/[controller]")]
public class ProductosController : ControllerBase
{
    // Cualquier usuario autenticado puede ver productos
    [HttpGet]
    [Authorize]                         // = tiene JWT válido
    public async Task<IActionResult> GetAll() { ... }

    // Solo administradores pueden crear
    [HttpPost]
    [Authorize(Roles = "admin")]        // = JWT + rol admin
    public async Task<IActionResult> Crear([FromBody] Producto p) { ... }

    // Solo administradores pueden eliminar
    [HttpDelete("{id}")]
    [Authorize(Roles = "admin")]        // = JWT + rol admin
    public async Task<IActionResult> Eliminar(int id) { ... }

    // Cualquiera puede ver el catálogo público
    [HttpGet("publico")]
    [AllowAnonymous]                    // = no necesita JWT
    public async Task<IActionResult> GetPublico() { ... }
}
```

Atributo	¿Necesita JWT?	¿Necesita rol?	Si falla retorna
[AllowAnonymous]	No	No	—
[Authorize]	Sí	No	401 Unauthorized
[Authorize(Roles = "admin")]	Sí	Sí (admin)	403 Forbidden

> **401 vs 403:** si el usuario no envía JWT (o es inválido), la API retorna **401** — "no sé quién sos". Si envía un JWT válido pero su rol no tiene permiso, retorna **403** — "sé quién sos, pero no podés hacer esto". El interceptor de errores de Angular que ya implementamos maneja ambos casos.

05 · Frontend: decodificar el JWT en Angular

El JWT que la API nos devuelve tiene tres partes separadas por puntos. La segunda parte (el *payload*) contiene los claims codificados en Base64. Para leer el rol del usuario no necesitamos instalar ninguna librería — podemos decodificar el payload con una función simple de JavaScript.

Función para decodificar el payload del JWT

services/auth.service.ts

```
// Función auxiliar para decodificar el payload del JWT
private decodeToken(token: string): any {
    try {
        const payload = token.split('.')[1];   // segunda parte
        const decoded = atob(payload);          // Base64 -> texto
        return JSON.parse(decoded);             // texto -> objeto
    } catch {
        return null;
    }
}
```

La función `atob()` es nativa del navegador — decodifica un string en Base64 a texto plano. No estamos *validando* la firma del JWT (eso lo hace la API), solo estamos *leyendo* su contenido para saber qué rol tiene el usuario.

Nuevos métodos en el AuthService

services/auth.service.ts — agregar a la clase

```
// Obtener el rol del usuario actual
getRol(): string | null {
    const token = this.getToken();
    if (!token) return null;

    const payload = this.decodeToken(token);
    if (!payload) return null;

    // El claim de rol en .NET se serializa con esta clave:
    return payload[
        "http://schemas.microsoft.com/ws/2008/06/identity/claims/role"
    ] || null;
}

// Verificar si el usuario tiene un rol específico
hasRole(role: string): boolean {
    return this.getRol() === role;
}

// Verificar si es administrador (atajo útil)
isAdmin(): boolean {
    return this.hasRole('admin');
}
```

> **¿Por qué la clave del claim es tan larga?** Cuando .NET serializa un `ClaimTypes.Role` al JWT, usa la URI completa del estándar: `http://schemas.microsoft.com/ws/2008/06/identity/claims/role`. Si querés una clave más corta, podés usar `new Claim("role", usuario.Rol)` en el backend, pero entonces `[Authorize(Roles)]` no lo lee automáticamente.

06 · Frontend: guard de roles

Ya tenemos un `authGuard` que verifica si el usuario está autenticado (tiene un JWT). Ahora creamos un `roleGuard` que, además de verificar autenticación, comprueba que el usuario tenga el rol necesario para acceder a esa ruta.

guards/role.guard.ts

```
export const roleGuard: CanActivateFn = (route, state) => {
    const authService = inject(AuthService);
    const router = inject(Router);

    // 1. ¿Está autenticado?
    if (!authService.isAuthenticated()) {
        router.navigate(['/login']);
        return false;
    }

    // 2. ¿Tiene el rol requerido?
    const rolesPermitidos = route.data['roles'] as string[];
    const rolUsuario = authService.getRol();

    if (rolesPermitidos && rolUsuario &&
        rolesPermitidos.includes(rolUsuario)) {
        return true;   // tiene el rol -> puede pasar
    }

    // No tiene el rol -> redirigir
    router.navigate(['/sin-permiso']);
    return false;
};
```

El guard lee la propiedad `data['roles']` de la configuración de la ruta. Eso nos permite definir qué roles pueden acceder a cada ruta directamente en el archivo de rutas, sin crear un guard diferente para cada rol.

Aplicar el guard en las rutas

app.routes.ts

```
export const routes: Routes = [
    // Ruta pública — sin guard (= [AllowAnonymous])
    { path: 'login', component: LoginComponent },

    // Solo autenticados (= [Authorize])
    { path: 'productos', component: ProductosComponent,
      canActivate: [authGuard] },

    // Solo admin (= [Authorize(Roles = "admin")])
    { path: 'admin/productos', component: AdminProductosComponent,
      canActivate: [roleGuard],
      data: { roles: ['admin'] } },          // <-- roles permitidos

    // Admin o moderador
    { path: 'gestion', component: GestionComponent,
      canActivate: [roleGuard],
      data: { roles: ['admin', 'moderador'] } },

    { path: 'sin-permiso', component: SinPermisoComponent },
    { path: '', redirectTo: 'login', pathMatch: 'full' },
    { path: '**', redirectTo: 'login' }
];
```

> **Analogía con .NET:** el objeto `data: { roles: ['admin'] }` en la ruta de Angular cumple exactamente la misma función que `[Authorize(Roles = "admin")]` en el controlador de .NET. Ambos declaran *qué roles* tienen acceso. La diferencia es dónde se evalúa: el atributo de .NET protege el endpoint en el servidor, el guard de Angular protege la ruta en el navegador.

07 · Frontend: mostrar y ocultar elementos según el rol

Además de proteger rutas completas con guards, necesitamos controlar qué elementos ve cada usuario dentro de un mismo componente. Por ejemplo: el botón "Eliminar" solo debe aparecer si el usuario es administrador.

Inyectar AuthService en el componente

components/productos/productos.component.ts

```
@Component({
    standalone: true,
    imports: [CommonModule],
    selector: 'app-productos',
    templateUrl: './productos.component.html'
})
export class ProductosComponent {
    productos: any[] = [];

    constructor(public authService: AuthService) {}

    // ... cargar productos, etc.
}
```

Observá que inyectamos el AuthService como `public`. Esto es necesario para poder usarlo directamente desde el template HTML. Si fuera `private`, el template no tendría acceso.

Usar *ngIf en el template

productos.component.html

```
<div *ngFor="let p of productos">
    <h3>{{ p.nombre }}</h3>
    <p>Precio: {{ p.precio | currency:'ARS' }}</p>

    <!-- Solo visible para administradores -->
    <button *ngIf="authService.isAdmin()" (click)="eliminar(p.id)">
        Eliminar
    </button>

    <!-- Visible para admin Y moderador -->
    <button *ngIf="authService.hasRole('admin') || authService.hasRole('moderador')"
            (click)="editar(p.id)">
        Editar
    </button>
</div>
```

> **Importante:** ocultar un botón con `*ngIf` no protege el endpoint. Un usuario avanzado podría hacer el request manualmente con Postman. Por eso la protección debe existir en **ambos lados**: `[Authorize(Roles)]` en el backend protege los datos, `*ngIf` en el frontend mejora la experiencia del usuario. Nunca confiar solo en el frontend.

08 · Frontend: navbar dinámico y componente Sin Permiso

Un caso de uso muy común: la barra de navegación muestra distintos links según el tipo de usuario. Un visitante no logueado ve "Login", un usuario logueado ve "Productos" y "Logout", y un administrador ve además un link a "Administración".

navbar.component.html

```
<nav>
    <a routerLink="/home">Inicio</a>

    <!-- Solo si NO está logueado -->
    <a *ngIf="!authService.isAuthenticated()" routerLink="/login">Iniciar sesión</a>

    <!-- Solo si está logueado -->
    <a *ngIf="authService.isAuthenticated()" routerLink="/productos">Productos</a>

    <!-- Solo si es admin -->
    <a *ngIf="authService.isAdmin()" routerLink="/admin/productos">Administración</a>

    <!-- Solo si está logueado -->
    <button *ngIf="authService.isAuthenticated()" (click)="logout()">Cerrar sesión</button>
</nav>
```

navbar.component.ts

```
@Component({
    standalone: true,
    imports: [CommonModule, RouterModule],
    selector: 'app-navbar',
    templateUrl: './navbar.component.html'
})
export class NavbarComponent {
    constructor(public authService: AuthService, private router: Router) {}

    logout(): void {
        this.authService.logout();
        this.router.navigate(['/login']);
    }
}
```

Componente Sin Permiso

Cuando el `roleGuard` redirige al usuario porque no tiene el rol necesario, necesitamos una página que le avise qué pasó. Es el equivalente visual del error 403 Forbidden.

components/sin-permiso/sin-permiso.component.ts

```
@Component({
    standalone: true,
    imports: [RouterModule],
    selector: 'app-sin-permiso',
    template: `
        <div class="sin-permiso">
            <h2>Acceso denegado</h2>
            <p>No tenés permisos para acceder a esta sección.</p>
            <a routerLink="/home">Volver al inicio</a>
        </div>`
})
export class SinPermisoComponent {}
```

09 · Flujo completo: todo junto

Veamos cómo todas las piezas trabajan juntas cuando un administrador se loguea e intenta acceder al panel de administración, y qué pasa cuando un usuario común intenta lo mismo.

Escenario A: el administrador accede

Paso	Capa	Qué sucede
1	**Usuario**	Ingresa email y contraseña en el formulario
2	**AuthService**	POST /api/auth/login con las credenciales
3	**API .NET**	Valida, genera JWT con claim role = admin
4	**AuthService**	Guarda el JWT en localStorage
5	**Usuario**	Navega a /admin/productos
6	**roleGuard**	isAuthenticated() → true. getRol() → "admin". roles incluye "admin" → true
7	**AdminProductosComponent**	Se activa y carga los datos
8	**Interceptor (auth)**	Agrega Authorization: Bearer al request
9	**API .NET**	[Authorize(Roles = "admin")] → rol es admin → 200 OK

Escenario B: el usuario común intenta acceder

Paso	Capa	Qué sucede
1-4	…	Login exitoso con JWT que tiene role = user
5	**Usuario**	Intenta navegar a /admin/productos
6	**roleGuard**	isAuthenticated() → true. getRol() → "user". roles NO incluye "user" → false
7	**roleGuard**	Redirige a /sin-permiso. El componente nunca se activa.

> **Doble protección:** incluso si un usuario avanzado manipulara el guard del frontend (modificando el código en el navegador), el backend tiene `[Authorize(Roles = "admin")]` que valida el claim real del JWT. Es imposible falsificar el claim sin la clave secreta del servidor. La seguridad real siempre está en el backend; el frontend solo mejora la experiencia.

10 · Resumen comparativo y conceptos clave

Backend vs Frontend

Responsabilidad	.NET (Backend)	Angular (Frontend)
Almacenar el rol	Columna Rol en la tabla Usuario	Viene dentro del JWT (claim)
Incluir rol en el JWT	new Claim(ClaimTypes.Role, rol)	No aplica (lo hace el backend)
Leer el rol del JWT	[Authorize(Roles)] lo lee automáticamente	authService.getRol() decodifica el payload
Proteger recurso por rol	[Authorize(Roles = "admin")]	roleGuard + data: { roles: ["admin"] }
Acceso público	[AllowAnonymous]	Ruta sin guard
Recurso no autorizado	Retorna 403 Forbidden	Redirige a /sin-permiso
Ocultar UI según rol	No aplica (el backend no tiene UI)	*ngIf="authService.isAdmin()"

Resumen de conceptos clave

Concepto	Definición rápida
JWT	String codificado en Base64 con tres partes: Header, Payload y Firma
Header (JWT)	Primera parte del token — indica el algoritmo de firma (ej: HS256)
Payload (JWT)	Segunda parte del token — contiene los claims del usuario
Firma (JWT)	Tercera parte — resultado de firmar Header + Payload con la clave secreta
Bearer	Prefijo del header Authorization que indica que se porta un token JWT
Autenticación	Verificar quién es el usuario (login + JWT)
Autorización	Verificar qué puede hacer el usuario (roles + permisos)
Claim	Par clave-valor dentro del JWT que afirma algo sobre el usuario
ClaimTypes.Role	Claim estándar de .NET para el rol del usuario
[Authorize(Roles)]	Atributo de .NET que restringe un endpoint a roles específicos
decodeToken()	Función que decodifica el payload Base64 del JWT
getRol()	Método del AuthService que extrae el rol del JWT

Concepto	Definición rápida
hasRole()	Método que verifica si el usuario tiene un rol específico
isAdmin()	Atajo para verificar si el rol es admin
roleGuard	Guard que verifica autenticación Y rol antes de activar una ruta
route.data['roles']	Array de roles permitidos definido en la configuración de la ruta
***nglf + isAdmin()**	Mostrar/ocultar elementos del template según el rol